Data Warehousing + Data Modeling + Snowflake SQL Lab

This module is designed as a **teaching + hands-on lab guide**, with explanations, mental models, SQL, illustrations, exercises, expected outputs, cheat sheets, and a complete Snowflake implementation.

---

1. Learning Roadmap

DATA WAREHOUSING

1 DATA WAREHOUSE ARCHITECTURE

STAGING LAYER

Raw data landing from multiple sources

- Operational Databases
- Flat Files
- APIs
- Streaming Data

- Extract data
- Validate & Cleanse
- Standardize formats

INTEGRATION LAYER

Integrated, cleansed, conformed data

Enterprise Data Warehouse (EDW)

Data Marts

- Transform & Integrate
- Apply Business Rules
- Maintain History

ACCESS LAYER

Deliver data for analytics and decision making

- BI Reports
- Dashboards
- Ad-hoc Queries
- Data Science & ML
- Applications

- Aggregate & Serve
- Secure & Govern
- Performance Optimize

2 EDW vs DATA MART

Aspect	Enterprise Data Warehouse (EDW)	Data Mart
Scope	Enterprise-wide, integrated data across all departments	Department/subject specific subset of EDW
Purpose	Single source of truth for the entire organization	Focused analysis for specific business needs
Data Volume	Very large	Smaller than EDW
Users	All enterprise users	Departmental users
Data Sources	Multiple sources across organization	EDW or specific sources
Update Frequency	Batch (daily/periodic)	Can be batch or real-time
Examples	Customer, Product, Finance EDW	Sales Mart, HR Mart, Marketing Mart

3 ETL vs ELT

ETL (Extract, Transform, Load)

Extract → Transform → Load

Transformations happen in ETL tool before loading to the warehouse

- Transform outside the warehouse
- Suitable for smaller data volumes
- Higher load on ETL server
- Data is modeled before load

ELT (Extract, Load, Transform)

Extract → Load → Transform

Data is loaded first into the warehouse and then transformed

- Leverages warehouse compute power
- Scalable for large data volumes
- Faster loading
- Transform using SQL in warehouse

4 BATCH vs REAL-TIME

BATCH PROCESSING

- Data collected over time
- Processed at scheduled intervals (e.g., hourly, daily)
- Suitable for historical reporting, dashboards
- Lower complexity

REAL-TIME PROCESSING

- Data processed as it arrives
- Immediate insights
- Used for monitoring, fraud detection, recommendations
- Higher complexity & cost

5 CLOUD DATA WAREHOUSING

snowflake

- Multi-cloud (AWS, Azure, GCP)
- Separation of storage & compute
- Scalable & elastic
- Supports semi-structured data
- Time Travel & Zero-copy cloning
- Secure Data Sharing

Google BigQuery

- Fully managed serverless
- Pay-per-query (on-demand pricing)
- Highly scalable & fast
- Built-in ML & GIS capabilities
- Seamless integration with Google Cloud ecosystem

amazon REDSHIFT

- AWS fully managed
- Petabyte-scale data warehousing
- High performance with columnar storage
- Integrates with AWS services
- Concurrency scaling

databricks

- Lakehouse architecture
- Built on Apache Spark
- Unifies Data Engineering, Data Science & ML
- Supports Delta Lake (ACID transactions)
- Open & multi-cloud

6 DATA MODELING

FACT TABLES

Store measurable business processes (metrics)

sales_fact

- order_id (FK)
- product_id (FK)
- customer_id (FK)
- date_id (FK)
- quantity
- sales_amount
- cost_amount

DIMENSION TABLES

Provide context and descriptive attributes

dim_customer

- customer_key (PK)
- customer_id (NK)
- customer_name
- email
- city
- state
- country
- ...

GRAIN

Level of detail in a fact table (one row represents)

Examples:

- Daily sales of a product in a store
- Transaction level sales
- Monthly aggregated sales

Grain defines the uniqueness of a row in fact table.

SURROGATE KEYS

System generated keys with no business meaning

dim_product

- product_key (PK)
- product_id (NK)
- product_name
- brand
- category
- sub_category
- ...

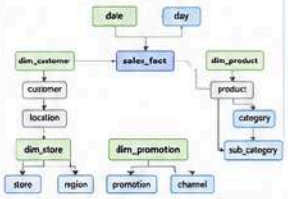
PK = Surrogate Key
NK = Natural Key

STAR SCHEMA

Denormalized dimensions connected to fact table

SNOWFLAKE SCHEMA

Normalized dimensions (split into related tables)



SCD (SLOWLY CHANGING DIMENSIONS)

Type 1: Overwrite

- Overwrite old data with new data
- No history kept

Old Data → New Data

Type 2: Add New Row

- Insert new row with new surrogate key
- Keep history with effective dates

Old Row (key=1) (Effective: 01-Jan-23 to 31-Dec-23)

New Row (key=2) (Effective: 01-Jan-24 to Current)

Type 3: Add New Column

- Add new column for previous value
- Keep limited history

Current Value	Previous Value	Start Date
Value B	Value A	01-Jan-24

NORMALIZATION vs DENORMALIZATION

Normalization (OLTP Focus)

- Minimize redundancy
- More tables & joins
- Save storage space
- Complex queries
- Used in operational systems

Denormalization (OLAP Focus)

- Reduce joins
- Improve query performance
- Increase redundancy
- More storage
- Used in data warehouses

DATA WAREHOUSING CHEAT SHEET

Key Terms

- EDW – Enterprise Data Warehouse
- ETL – Extract, Transform, Load
- ELT – Extract, Load, Transform
- SCD – Slowly Changing Dimension
- PK – Primary Key
- FK – Foreign Key
- Grain – Level of Detail

Common SQL in DWH

- SELECT – Query data
- JOIN – Combine tables
- GROUP BY – Aggregate data
- WHERE – Filter data
- ORDER BY – Sort data
- DISTINCT – Unique rows
- HAVING – Filter aggregates

Common Aggregations

- SUM() – Total
- AVG() – Average
- COUNT() – Count
- MIN() – Minimum
- MAX() – Maximum

DWH Best Practices

- Define the right grain
- Use surrogate keys
- Handle SCD properly
- Audit & data quality checks
- Partition large fact tables
- Use meaningful naming
- Document everything

Typical Data Flow

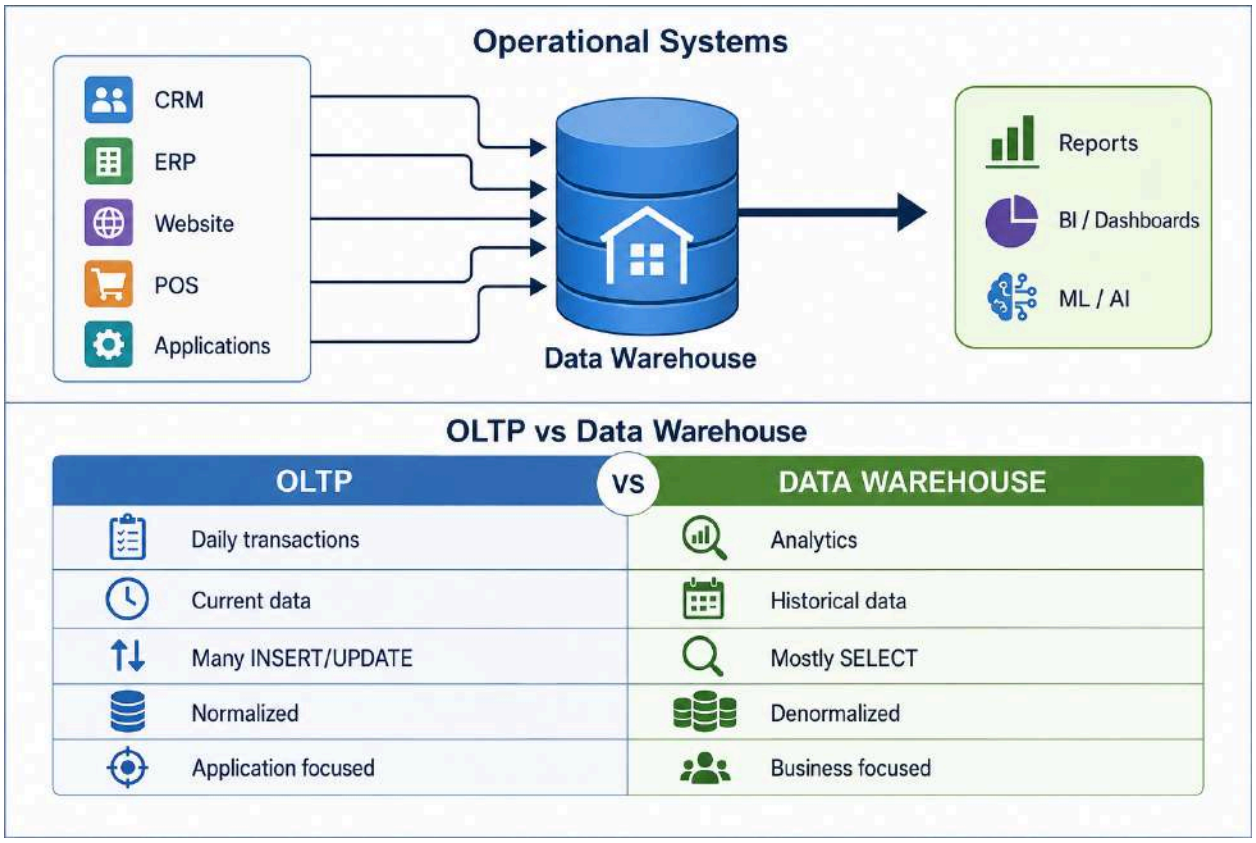
Source Systems → Staging (Extract) → Integration (Transform) → Access (Analytics)

2. What Is a Data Warehouse?

A **Data Warehouse (DWH)** is a centralized system designed to store integrated historical data for:

- Reporting
- Analytics
- Business intelligence
- Dashboards
- Decision-making

A simple mental model:



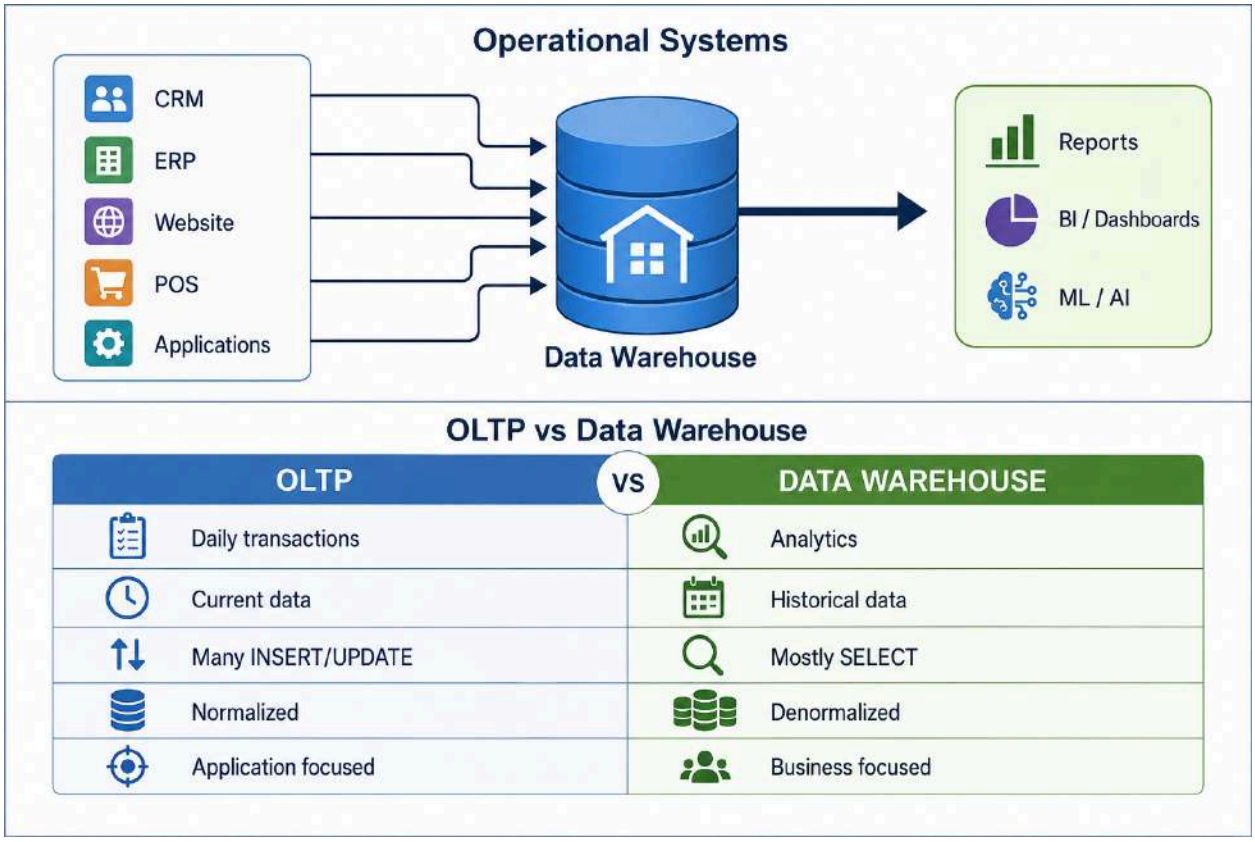
3. Data Warehouse Architecture

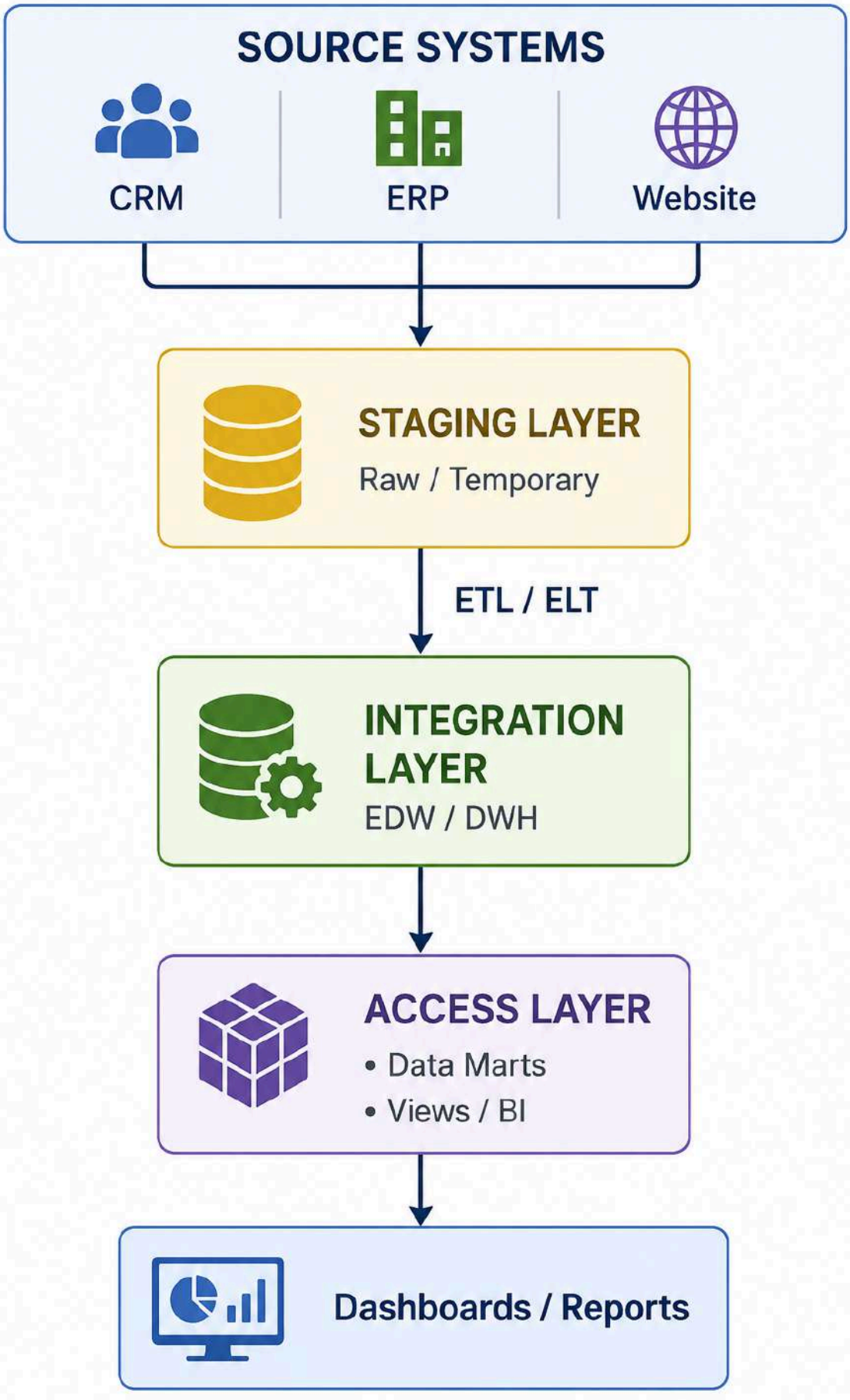
A basic architecture contains three major layers:

3.1 Staging Layer

The staging layer is where incoming data is temporarily landed.

Example:

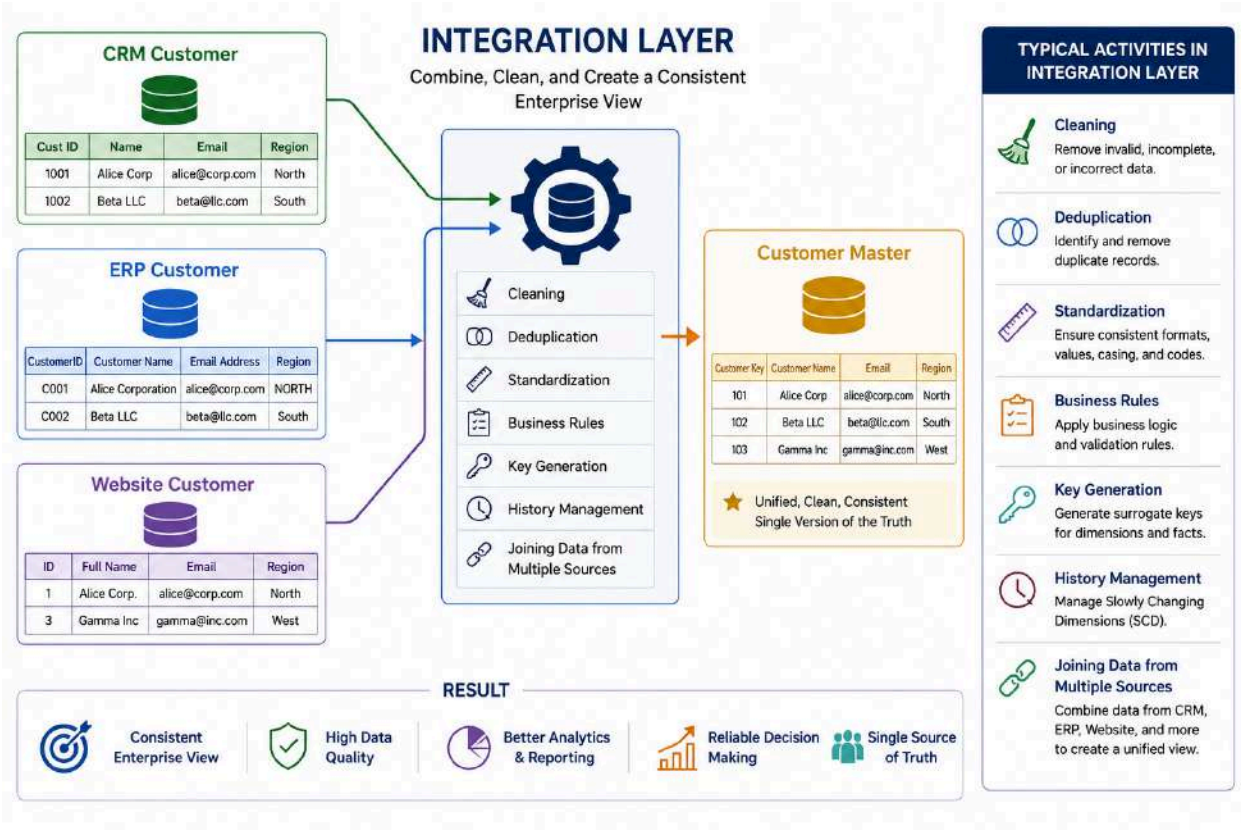



SOURCE SYSTEMS
CRM
ERP
Website
STAGING LAYER
Raw / Temporary
ETL / ELT
INTEGRATION LAYER
EDW / DWH
ACCESS LAYER
• Data Marts
• Views / BI
Dashboards / Reports

4. Integration Layer

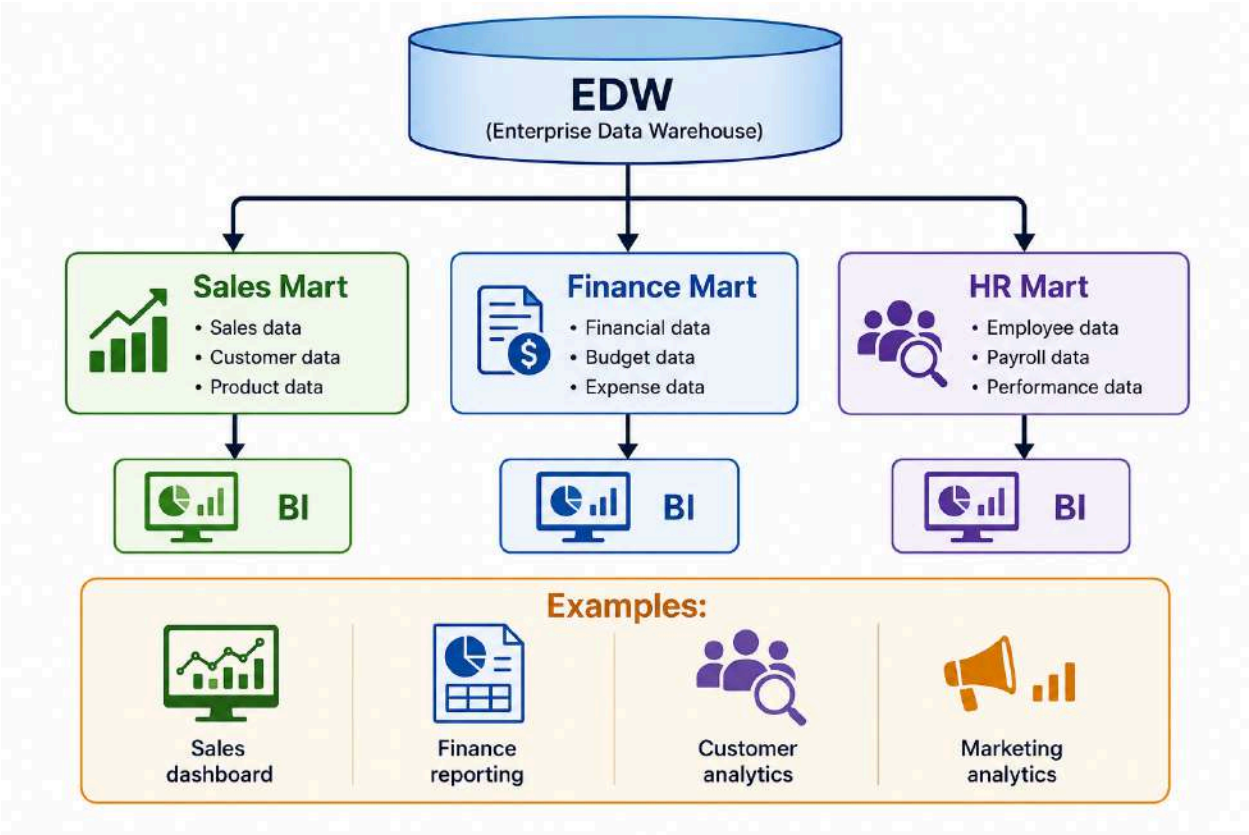
The integration layer creates a **consistent enterprise view of data**.

Example:



5. Access Layer

The access layer is optimized for consumers.

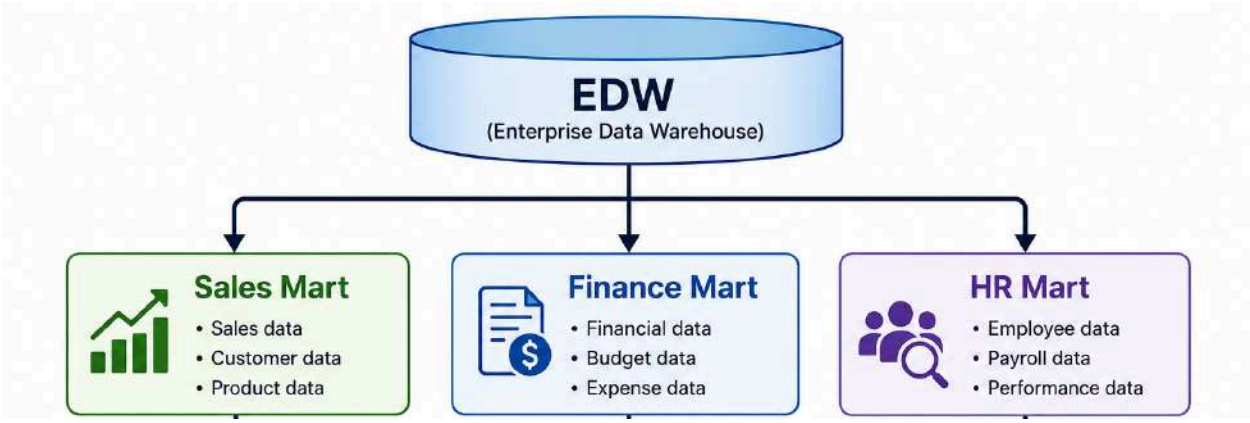


---

6. Enterprise Data Warehouse vs Data Mart

Enterprise Data Warehouse

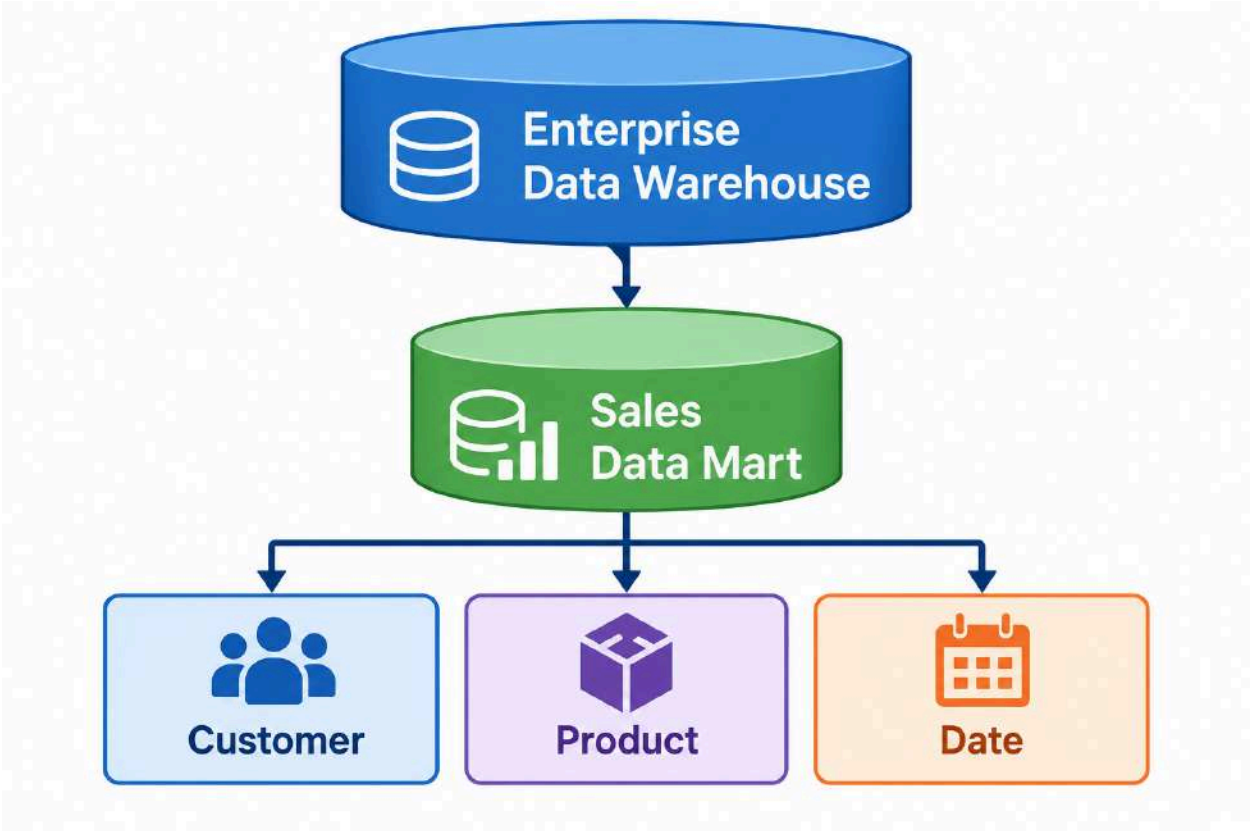
An **EDW** is organization-wide.



It normally integrates data across multiple business areas.

Data Mart

A **Data Mart** focuses on a particular business area.



Feature	EDW	Data Mart
Scope	Enterprise	Department/business area
Size	Larger	Smaller

Users	Many departments	Specific department
Example	Company-wide warehouse	Sales mart
Complexity	Higher	Lower

---

7. ETL vs ELT

ETL

Extract → Transform → Load

SOURCE
  |
  ▼
EXTRACT
  |
  ▼
TRANSFORM
  |
  ▼
LOAD
  |
  ▼
WAREHOUSE

Transformation happens before loading into the warehouse.

ELT

Extract → Load → Transform

SOURCE
  |
  ▼
EXTRACT
  |
  ▼
LOAD

Modern cloud warehouses make ELT attractive because transformation can be performed using warehouse compute.

Easy memory trick

ETL = Transform before warehouse

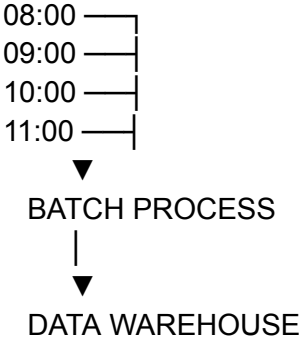
ELT = Transform inside warehouse

---

8. Batch vs Real-Time Data Warehousing

Batch Processing

Data is collected and processed periodically.



Examples:

- Daily sales load
- Nightly finance processing
- Monthly reporting

Batch Lab

Suppose we receive:

sales_2025_01_01.csv
sales_2025_01_02.csv
sales_2025_01_03.csv

We can load them once per day.

CSV Files
  |
  ▼
STAGE
  |
  ▼
COPY INTO
  |
  ▼
FACT_SALES

Snowflake's `COPY INTO <table>` loads staged files into an existing table.

---

9. Batch Data Lab — Snowflake

Objective

Build a simple batch pipeline:

CSV
↓
STAGE
↓
STAGING TABLE
↓
DIMENSION / FACT
↓
ANALYTICS

Step 1 — Create database

CREATE DATABASE SALES_DWH;

Step 2 — Create schemas

CREATE SCHEMA SALES_DWH.STAGING;

CREATE SCHEMA SALES_DWH.DIMENSIONS;

CREATE SCHEMA SALES_DWH.FACTS;

CREATE SCHEMA SALES_DWH.MART;

Snowflake databases contain schemas, while schemas contain database objects such as tables and views.

Step 3 — Select database

USE DATABASE SALES_DWH;

Step 4 — Select schema

USE SCHEMA STAGING;

Snowflake supports `USE DATABASE` and `USE SCHEMA` to establish the current namespace for subsequent SQL statements.

---

10. Batch Loading with a Stage

Create a file format:

```
CREATE OR REPLACE FILE FORMAT SALES_CSV_FORMAT
TYPE = CSV
FIELD_DELIMITER = ','
SKIP_HEADER = 1;
```

Create an internal stage:

```
CREATE OR REPLACE STAGE SALES_STAGE
FILE_FORMAT = SALES_CSV_FORMAT;
```

Snowflake supports internal stages for storing files before loading them into tables.

For local files, a client such as SnowSQL/Snowflake CLI can use `PUT` to upload files to an internal stage.

PUT file:///path/sales.csv @SALES_STAGE;

Then:

```
COPY INTO STG_SALES
FROM @SALES_STAGE
FILE_FORMAT = (FORMAT_NAME = SALES_CSV_FORMAT);
```

---

11. Data Modeling

Data modeling means deciding:

> **What data should we store, at what grain, and how should the entities relate?**

The most important concepts are:

```
Data Modeling
    │
    ├── Grain
    ├── Fact
    ├── Dimension
    ├── Keys
    ├── Relationships
    ├── Star Schema
    ├── Snowflake Schema
    └── SCD
```

---

12. Fact Table

A **fact table** stores measurable business events.

Examples:

- Sales
- Orders
- Payments
- Shipments
- Website visits

Example:

FACT_SALES

sales_key

date_key
customer_key
product_key
quantity
sales_amount

The fact table usually contains:

Foreign keys

date_key
customer_key
product_key

Measures

quantity
sales_amount

---

13. Dimension Table

A dimension provides descriptive context.

Example:

DIM_CUSTOMER

customer_key
customer_name
region
segment

The fact says:

> "What happened?"

The dimension says:

> "Who / what / when / where?"

---

14. Fact vs Dimension

Fact	Dimension
Measures events	Describes entities
Numeric measures	Descriptive attributes
Large table	Usually smaller
Foreign keys	Primary/surrogate key
Sales amount	Customer name
Quantity	Product category

Memory trick

FACT = WHAT HAPPENED?

DIMENSION = WHO / WHAT / WHEN / WHERE?

---

15. Grain

Grain is one of the most important concepts in dimensional modeling.

Grain answers:

> **What does one row in the fact table represent?**

For example:

One row = one product sold to one customer
on one date

Then:

FACT_SALES
---
1 row = 1 sales transaction line

If you don't define grain first, your fact table can become inconsistent.

Example

Bad:

One row = sometimes an order
One row = sometimes a product
One row = sometimes a customer

Good:

One row = one product line in one sales transaction

---

16. Surrogate Keys

A surrogate key is an artificial key generated by the data warehouse.

Example:

CUSTOMER SOURCE ID	CUSTOMER_KEY
1001	1
1002	2
1003	3

The source's natural identifier might be:

customer_id = 1001

The warehouse can use:

customer_key = 1

Why?

Because source systems can change.

CRM customer_id = 1001
ERP customer_id = C1001

The warehouse can maintain one common surrogate key.

---

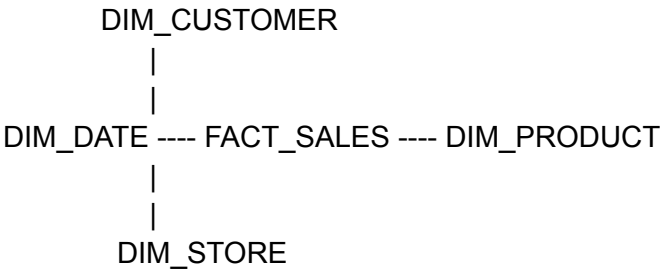
17. Natural Key vs Surrogate Key

Natural Key	Surrogate Key
Comes from business/source	Created by warehouse
Has business meaning	Usually no business meaning
Can change	Stable
Example: email/customer ID	Example: customer_key = 101

---

18. Star Schema

A Star Schema has:



The fact table is in the center.

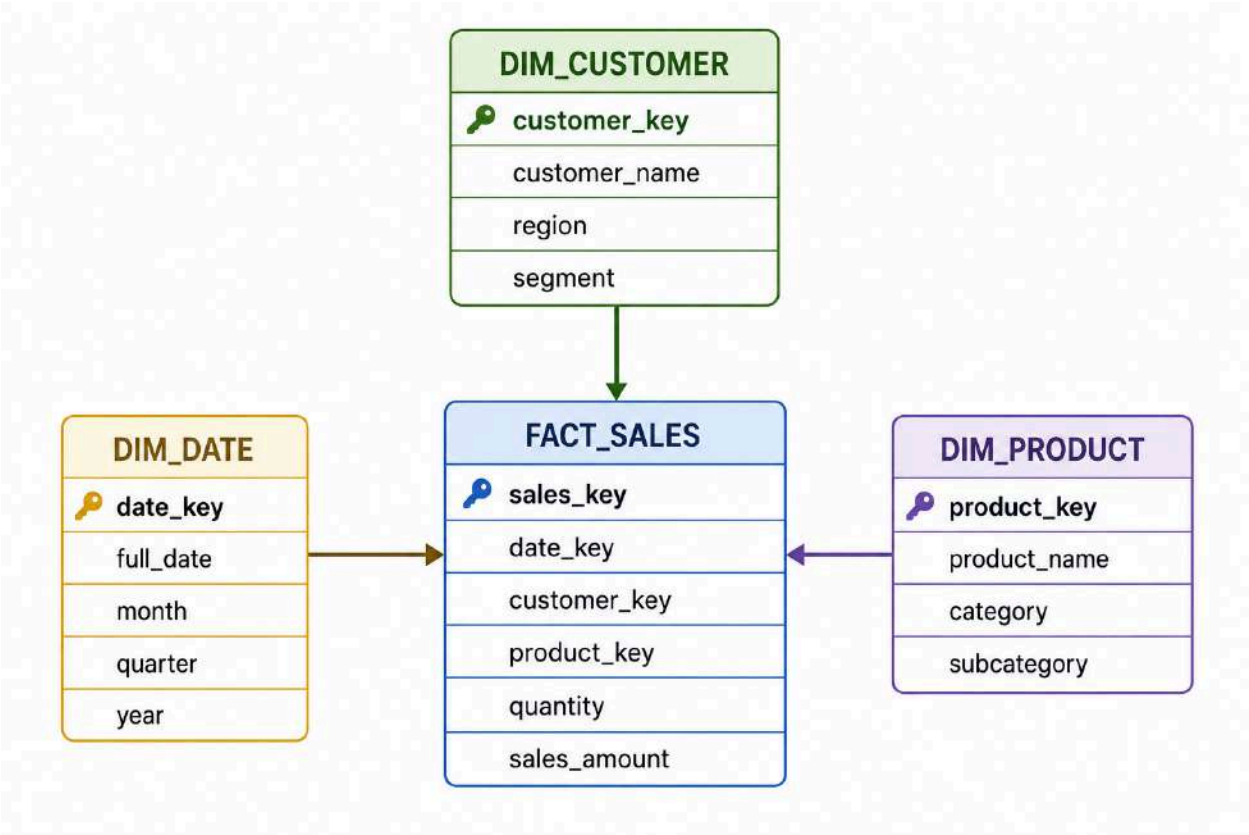
Dimensions surround it.

Hence:

STAR SCHEMA

---

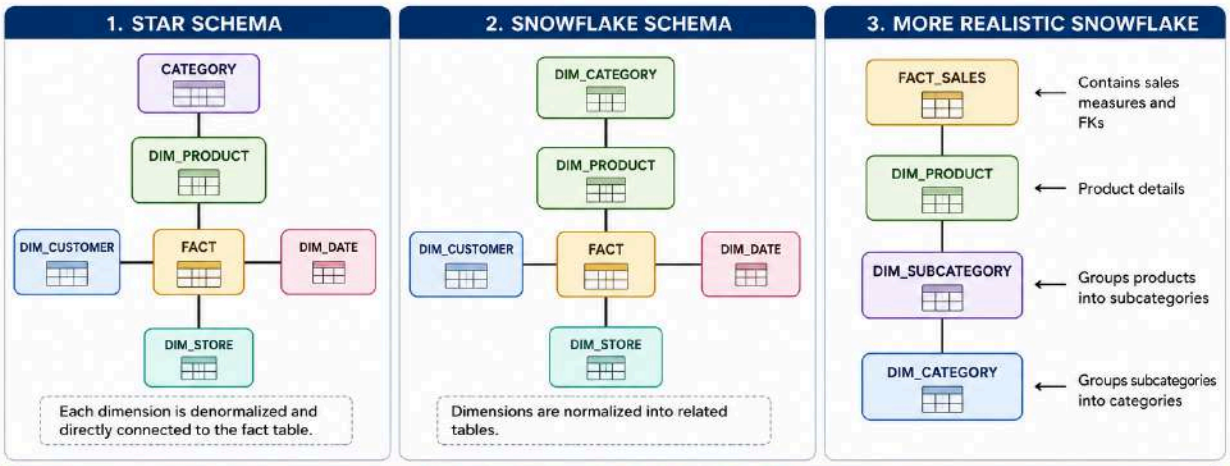
19. Star Schema Example



---

20. Snowflake Schema

A Snowflake Schema normalizes dimensions.



4. COMPARISON: STAR VS SNOWFLAKE

ASPECT	STAR SCHEMA	SNOWFLAKE SCHEMA
Structure	Denormalized dimensions	Normalized dimensions
Joins	Fewer joins	More joins
Query Complexity	Simpler queries	More complex queries
Performance	Faster / simple BI queries	Slower due to more joins
Redundancy	More redundancy	Less redundancy
Ease for Analysts	Easier for analysts	More complex model
Storage	More storage	Less storage
Use Case	Best for reporting & BI	Better for data consistency & detailed analysis

---

21. Normalization vs Denormalization

Normalization

Reduce redundancy.

PRODUCT
  |
  +-- CATEGORY
  |
  +-- SUBCATEGORY

Advantages:

- Less duplication
- Better consistency
- Easier updates

Disadvantages:

- More joins
- More complex analytical queries

Denormalization

Keep related attributes together.

DIM_PRODUCT

product
subcategory
category
category_description

Advantages:

- Fewer joins
- Easier analytics
- Often convenient for BI

Disadvantage:

- More repeated information

---

22. Slowly Changing Dimensions

Customer information can change.

Example:

Customer: Alice Corp
Region: North

Later:

Customer: Alice Corp
Region: South

How should the warehouse handle this?

That's the purpose of **SCD — Slowly Changing Dimensions**.

---

23. SCD Type 1

Type 1 **overwrites** the old value.

Before:

customer_key | customer | region
1 | Alice | North

After:

customer_key | customer | region
1 | Alice | South

Old value is lost.

Use when:

Historical changes are not important.

---

24. SCD Type 2

Type 2 preserves history.

Before:

customer_key | customer | region | current
101 | Alice | North | Y

After region changes:

customer_key | customer | region | current
101 | Alice | North | N
205 | Alice | South | Y

Often additional columns are used:

effective_start_date
effective_end_date
is_current

Example:

```
+--------------+-------+--------+------------+------------+-----------+
| customer_key | name  | region | start_date | end_date   | is_current|
+--------------+-------+--------+------------+------------+-----------+
| 101          | Alice | North  | 2025-01-01 | 2025-06-30 | N         |
| 205          | Alice | South  | 2025-07-01 | NULL       | Y         |
+--------------+-------+--------+------------+------------+-----------+
```

---

25. SCD Type 3

Type 3 stores current and previous values.

customer_key
customer_name
current_region
previous_region

Example:

1 | Alice | South | North

Only limited history is maintained.

---

26. SCD Comparison

Type	Technique	History
Type 1	Overwrite	No
Type 2	New row	Full history
Type 3	Previous-value column	Limited history

Memory trick

TYPE 1 → REPLACE

TYPE 2 → NEW ROW

TYPE 3 → NEW COLUMN

---

27. Snowflake Multi-Database Architecture

For your lab, use multiple databases to demonstrate enterprise organization.

```
SNOWFLAKE ACCOUNT
      │
      ├── RAW_DB
      │     └── RAW
      │
      ├── DWH_DB
      │     ├── STAGING
      │     ├── DIMENSIONS
      │     └── FACTS
      │
      └── MART_DB
            └── SALES
```

This demonstrates a very important Snowflake concept:

```
DATABASE
  ↓
SCHEMA
  ↓
TABLE
```

For example:

DWH_DB.DIMENSIONS.DIM_CUSTOMER

means:

```
Database = DWH_DB
Schema   = DIMENSIONS
Table    = DIM_CUSTOMER
```

---

28. Complete Snowflake Lab

Lab Objective

Build:

RAW_DB
  ↓
DWH_DB
  ↓
MART_DB

with:

RAW
 ├── customers
 ├── products
 └── sales

DWH
 ├── dim_customer
 ├── dim_product
 ├── dim_date
 └── fact_sales

MART
 └── sales_summary

---

29. Step 1 — Create Databases

CREATE DATABASE IF NOT EXISTS RAW_DB;

CREATE DATABASE IF NOT EXISTS DWH_DB;

CREATE DATABASE IF NOT EXISTS MART_DB;

Snowflake supports database creation and database/schema organization natively.

---

30. Step 2 — Create Schemas

```
CREATE SCHEMA IF NOT EXISTS RAW_DB.RAW;

CREATE SCHEMA IF NOT EXISTS DWH_DB.STAGING;

CREATE SCHEMA IF NOT EXISTS DWH_DB.DIMENSIONS;

CREATE SCHEMA IF NOT EXISTS DWH_DB.FACTS;

CREATE SCHEMA IF NOT EXISTS MART_DB.SALES;
```

---

31. Step 3 — Create Raw Customer Table

```
CREATE OR REPLACE TABLE RAW_DB.RAW.CUSTOMERS (
   CUSTOMER_ID INT,
   CUSTOMER_NAME VARCHAR(100),
   REGION VARCHAR(50),
   SEGMENT VARCHAR(50)
);
```

Insert data:

```
INSERT INTO RAW_DB.RAW.CUSTOMERS
VALUES
(1001, 'Alice Corp', 'North', 'Enterprise'),
(1002, 'Beta LLC', 'South', 'SMB'),
(1003, 'Gamma Inc', 'West', 'Enterprise');
```

---

32. Step 4 — Create Raw Product Table

```
CREATE OR REPLACE TABLE RAW_DB.RAW.PRODUCTS (
   PRODUCT_ID INT,
   PRODUCT_NAME VARCHAR(100),
   CATEGORY VARCHAR(50),
   SUB_CATEGORY VARCHAR(50)
);
```

Insert:

```
INSERT INTO RAW_DB.RAW.PRODUCTS
VALUES
(501, 'Laptop Pro', 'Electronics', 'Computers'),
(502, 'Office Chair', 'Furniture', 'Chairs'),
(503, 'Monitor X', 'Electronics', 'Monitors');
```

---

33. Step 5 — Create Raw Sales Table

```
CREATE OR REPLACE TABLE RAW_DB.RAW.SALES (
    SALES_ID INT,
    SALE_DATE DATE,
    CUSTOMER_ID INT,
    PRODUCT_ID INT,
    QUANTITY INT,
    SALES_AMOUNT DECIMAL(12,2)
);
```

Insert:

```
INSERT INTO RAW_DB.RAW.SALES
VALUES
(1, '2025-01-01', 1001, 501, 10, 15000.00),
(2, '2025-01-01', 1002, 502, 5, 1000.00),
(3, '2025-01-02', 1001, 503, 3, 1200.00),
(4, '2025-01-02', 1003, 501, 4, 6000.00),
(5, '2025-01-03', 1002, 502, 2, 400.00);
```

---

34. Step 6 — Create Dimension Tables

Customer Dimension

```
CREATE OR REPLACE TABLE DWH_DB.DIMENSIONS.DIM_CUSTOMER (
    CUSTOMER_KEY INT,
    CUSTOMER_ID INT,
    CUSTOMER_NAME VARCHAR(100),
    REGION VARCHAR(50),
    SEGMENT VARCHAR(50),
    PRIMARY KEY (CUSTOMER_KEY)
);
```

Load:

```
INSERT INTO DWH_DB.DIMENSIONS.DIM_CUSTOMER
SELECT
   ROW_NUMBER() OVER (ORDER BY CUSTOMER_ID) AS CUSTOMER_KEY,
   CUSTOMER_ID,
   CUSTOMER_NAME,
   REGION,
   SEGMENT
FROM RAW_DB.RAW.CUSTOMERS;
```

---

35. Product Dimension

```
CREATE OR REPLACE TABLE DWH_DB.DIMENSIONS.DIM_PRODUCT (
   PRODUCT_KEY INT,
   PRODUCT_ID INT,
   PRODUCT_NAME VARCHAR(100),
   CATEGORY VARCHAR(50),
   SUB_CATEGORY VARCHAR(50),
   PRIMARY KEY (PRODUCT_KEY)
);
```

Load:

```
INSERT INTO DWH_DB.DIMENSIONS.DIM_PRODUCT
SELECT
   ROW_NUMBER() OVER (ORDER BY PRODUCT_ID) AS PRODUCT_KEY,
   PRODUCT_ID,
   PRODUCT_NAME,
   CATEGORY,
   SUB_CATEGORY
FROM RAW_DB.RAW.PRODUCTS;
```

---

36. Date Dimension

```
CREATE OR REPLACE TABLE DWH_DB.DIMENSIONS.DIM_DATE (
   DATE_KEY INT,
   FULL_DATE DATE,
   DAY INT,
   MONTH INT,
```

```
    QUARTER INT,
    YEAR INT,
    PRIMARY KEY (DATE_KEY)
);
```

Load from sales dates:

```
INSERT INTO DWH_DB.DIMENSIONS.DIM_DATE
SELECT DISTINCT
    TO_NUMBER(TO_CHAR(SALE_DATE, 'YYYYMMDD')) AS DATE_KEY,
    SALE_DATE AS FULL_DATE,
    DAY(SALE_DATE) AS DAY,
    MONTH(SALE_DATE) AS MONTH,
    QUARTER(SALE_DATE) AS QUARTER,
    YEAR(SALE_DATE) AS YEAR
FROM RAW_DB.RAW.SALES;
```

---

37. Step 7 — Create Fact Table

```
CREATE OR REPLACE TABLE DWH_DB.FACTS.FACT_SALES (
    SALES_KEY INT,
    DATE_KEY INT,
    CUSTOMER_KEY INT,
    PRODUCT_KEY INT,
    QUANTITY_SOLD INT,
    SALES_AMOUNT DECIMAL(12,2),
    PRIMARY KEY (SALES_KEY)
);
```

---

38. Load the Fact Table

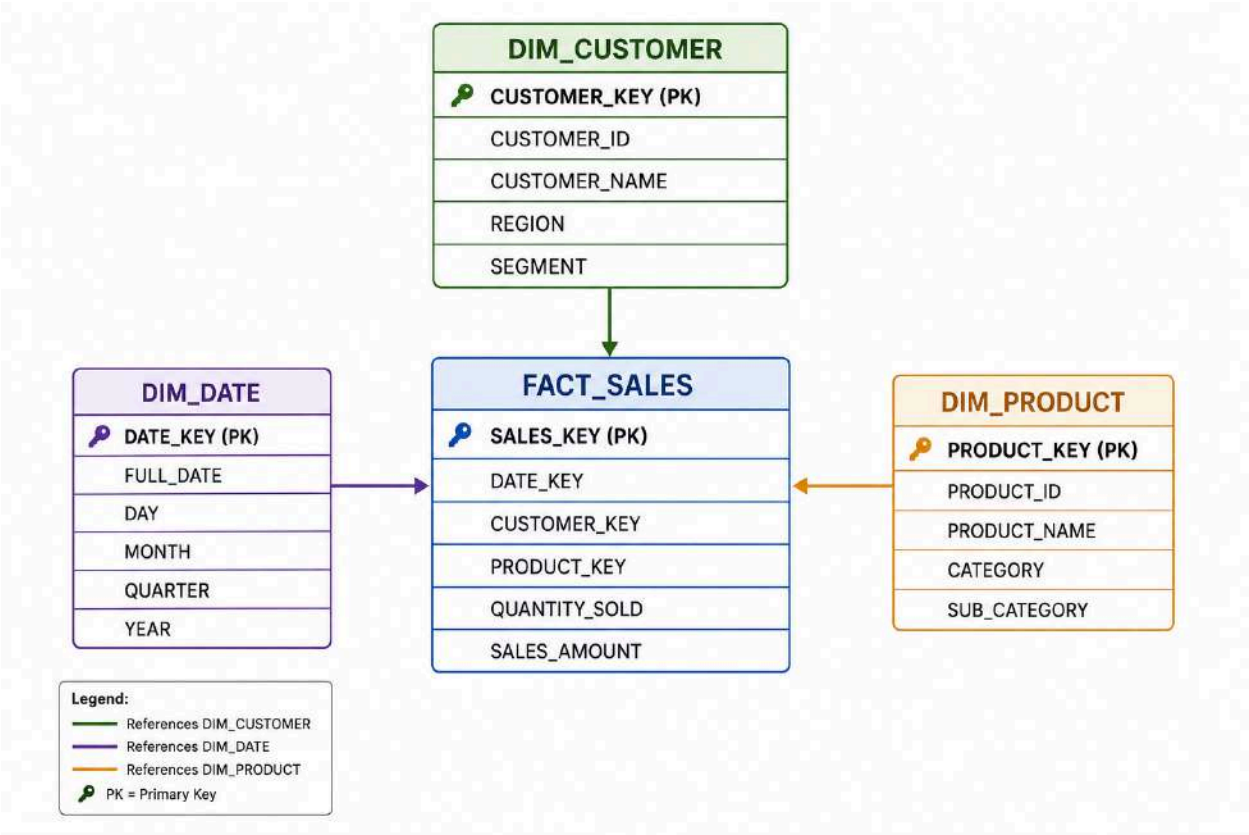
This is where the Star Schema is assembled.

```
INSERT INTO DWH_DB.FACTS.FACT_SALES
SELECT
    s.SALES_ID AS SALES_KEY,

    d.DATE_KEY,

    c.CUSTOMER_KEY,
```

```
    p.PRODUCT_KEY,

    s.QUANTITY,

    s.SALES_AMOUNT

FROM RAW_DB.RAW.SALES s

JOIN DWH_DB.DIMENSIONS.DIM_DATE d
    ON s.SALE_DATE = d.FULL_DATE

JOIN DWH_DB.DIMENSIONS.DIM_CUSTOMER c
    ON s.CUSTOMER_ID = c.CUSTOMER_ID

JOIN DWH_DB.DIMENSIONS.DIM_PRODUCT p
    ON s.PRODUCT_ID = p.PRODUCT_ID;
```

---

39. Final Star Schema



---

40. Step 8 — Analytical Queries

Query 1 — Total Sales

```
SELECT
   SUM(SALES_AMOUNT) AS TOTAL_SALES
FROM DWH_DB.FACTS.FACT_SALES;
```

Expected:

```
TOTAL_SALES
-----------
23600.00
```

---

41. Query 2 — Sales by Customer

```
SELECT
   c.CUSTOMER_NAME,
   SUM(f.SALES_AMOUNT) AS TOTAL_SALES
FROM DWH_DB.FACTS.FACT_SALES f
JOIN DWH_DB.DIMENSIONS.DIM_CUSTOMER c
   ON f.CUSTOMER_KEY = c.CUSTOMER_KEY
GROUP BY c.CUSTOMER_NAME
ORDER BY TOTAL_SALES DESC;
```

Expected:

```
Alice Corp    16200.00
Gamma Inc      6000.00
Beta LLC       1400.00
```

---

42. Query 3 — Sales by Category

```
SELECT
   p.CATEGORY,
   SUM(f.SALES_AMOUNT) AS TOTAL_SALES
FROM DWH_DB.FACTS.FACT_SALES f
JOIN DWH_DB.DIMENSIONS.DIM_PRODUCT p
   ON f.PRODUCT_KEY = p.PRODUCT_KEY
GROUP BY p.CATEGORY
ORDER BY TOTAL_SALES DESC;
```

Expected:

```
Electronics    22200.00
Furniture       1400.00
```

---

43. Query 4 — Daily Sales

```
SELECT
   d.FULL_DATE,
   SUM(f.SALES_AMOUNT) AS DAILY_SALES
FROM DWH_DB.FACTS.FACT_SALES f
JOIN DWH_DB.DIMENSIONS.DIM_DATE d
   ON f.DATE_KEY = d.DATE_KEY
GROUP BY d.FULL_DATE
ORDER BY d.FULL_DATE;
```

Expected:

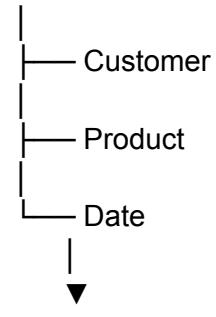
2025-01-01 16000.00
2025-01-02 7200.00
2025-01-03 400.00

---

44. Query 5 — Customer + Product Analysis

```
SELECT
   c.CUSTOMER_NAME,
   p.PRODUCT_NAME,
   SUM(f.QUANTITY_SOLD) AS TOTAL_QUANTITY,
   SUM(f.SALES_AMOUNT) AS TOTAL_SALES
FROM DWH_DB.FACTS.FACT_SALES f

JOIN DWH_DB.DIMENSIONS.DIM_CUSTOMER c
   ON f.CUSTOMER_KEY = c.CUSTOMER_KEY

JOIN DWH_DB.DIMENSIONS.DIM_PRODUCT p
   ON f.PRODUCT_KEY = p.PRODUCT_KEY

GROUP BY
   c.CUSTOMER_NAME,
   p.PRODUCT_NAME

ORDER BY TOTAL_SALES DESC;
```

This demonstrates the real power of a Star Schema:



---

45. Create a Sales Data Mart

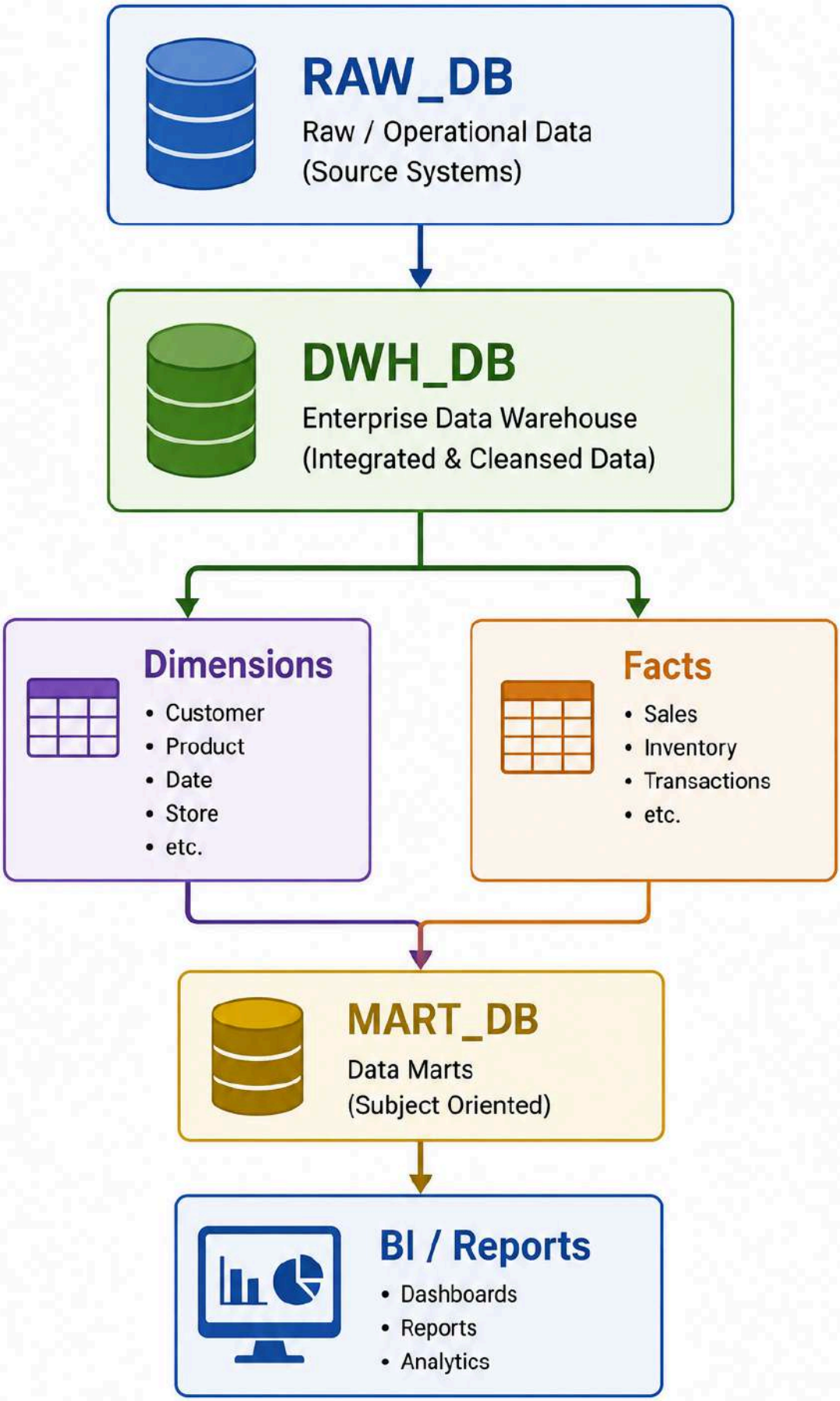
Now create a business-facing view/table.

```
CREATE OR REPLACE VIEW MART_DB.SALES.SALES_SUMMARY AS

SELECT
   d.FULL_DATE,
   c.CUSTOMER_NAME,
   c.REGION,
   c.SEGMENT,
   p.PRODUCT_NAME,
   p.CATEGORY,
   p.SUB_CATEGORY,
   f.QUANTITY_SOLD,
   f.SALES_AMOUNT

FROM DWH_DB.FACTS.FACT_SALES f

JOIN DWH_DB.DIMENSIONS.DIM_DATE d
   ON f.DATE_KEY = d.DATE_KEY

JOIN DWH_DB.DIMENSIONS.DIM_CUSTOMER c
   ON f.CUSTOMER_KEY = c.CUSTOMER_KEY

JOIN DWH_DB.DIMENSIONS.DIM_PRODUCT p
   ON f.PRODUCT_KEY = p.PRODUCT_KEY;
```

Query:

```
SELECT *
FROM MART_DB.SALES.SALES_SUMMARY;
```

Architecture now becomes:


RAW_DB
Raw / Operational Data
(Source Systems)
DWH_DB
Enterprise Data Warehouse
(Integrated & Cleansed Data)
Dimensions
• Customer
• Product
• Date
• Store
• etc.
Facts
• Sales
• Inventory
• Transactions
• etc.
MART_DB
Data Marts
(Subject Oriented)
BI / Reports
• Dashboards
• Reports
• Analytics

---

46. Cross-Database Query

This is an important Snowflake skill.

You can reference objects using:

DATABASE.SCHEMA.TABLE

Example:

```
SELECT *
FROM RAW_DB.RAW.CUSTOMERS;
```

Another:

```
SELECT *
FROM DWH_DB.DIMENSIONS.DIM_CUSTOMER;
```

And:

```
SELECT *
FROM MART_DB.SALES.SALES_SUMMARY;
```

This is why the three-part name is important:

DATABASE
.
SCHEMA
.
TABLE

---

47. SCD Type 1 Lab

Suppose Alice changes region.

Original:

1001 | Alice Corp | North

New source data:

1001 | Alice Corp | South

Type 1:

```
UPDATE DWH_DB.DIMENSIONS.DIM_CUSTOMER
SET REGION = 'South'
WHERE CUSTOMER_ID = 1001;
```

The old value `North` is lost.

---

48. SCD Type 2 Lab

Create a Type 2 dimension:

```
CREATE OR REPLACE TABLE DWH_DB.DIMENSIONS.DIM_CUSTOMER_SCD2 (
    CUSTOMER_KEY INT,
    CUSTOMER_ID INT,
    CUSTOMER_NAME VARCHAR(100),
    REGION VARCHAR(50),
    SEGMENT VARCHAR(50),
    EFFECTIVE_START_DATE DATE,
    EFFECTIVE_END_DATE DATE,
    IS_CURRENT BOOLEAN
);
```

Initial record:

```
INSERT INTO DWH_DB.DIMENSIONS.DIM_CUSTOMER_SCD2
VALUES
(
    1,
    1001,
    'Alice Corp',
    'North',
    'Enterprise',
    '2025-01-01',
    NULL,
    TRUE
);
```

Customer changes region.

Close the old record:

```
UPDATE DWH_DB.DIMENSIONS.DIM_CUSTOMER_SCD2
SET
    EFFECTIVE_END_DATE = '2025-06-30',
    IS_CURRENT = FALSE
WHERE CUSTOMER_ID = 1001
  AND IS_CURRENT = TRUE;
```

Insert the new version:

```
INSERT INTO DWH_DB.DIMENSIONS.DIM_CUSTOMER_SCD2
VALUES
(
    2,
    1001,
    'Alice Corp',
    'South',
    'Enterprise',
    '2025-07-01',
    NULL,
    TRUE
);
```

Now:

CUSTOMER_ID = 1001

VERSION 1
North
2025-01-01 → 2025-06-30

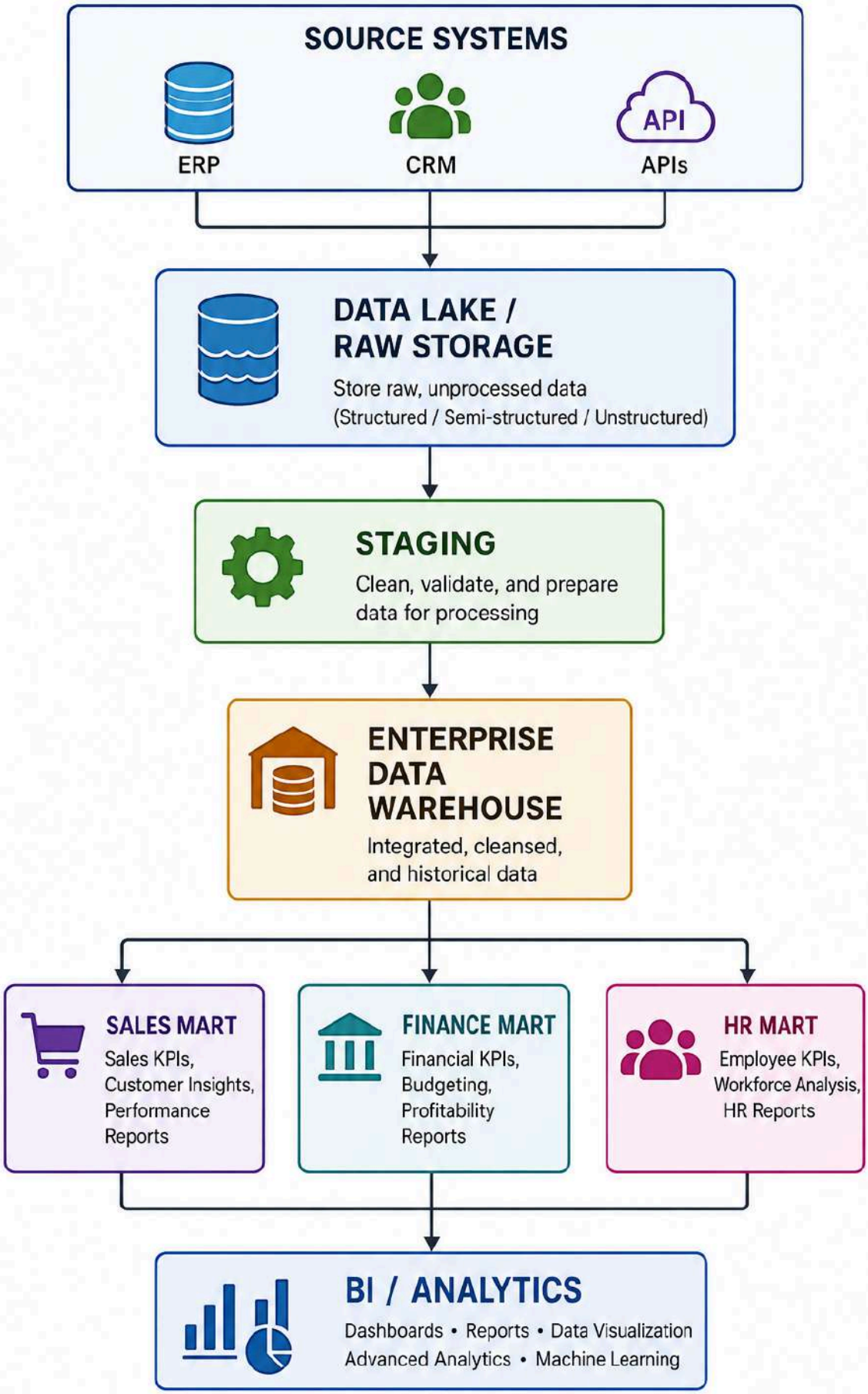
VERSION 2
South
2025-07-01 → current

This is the fundamental Type 2 pattern.

---

49. Advanced Architecture: Modern Cloud DWH

A more complete architecture looks like:

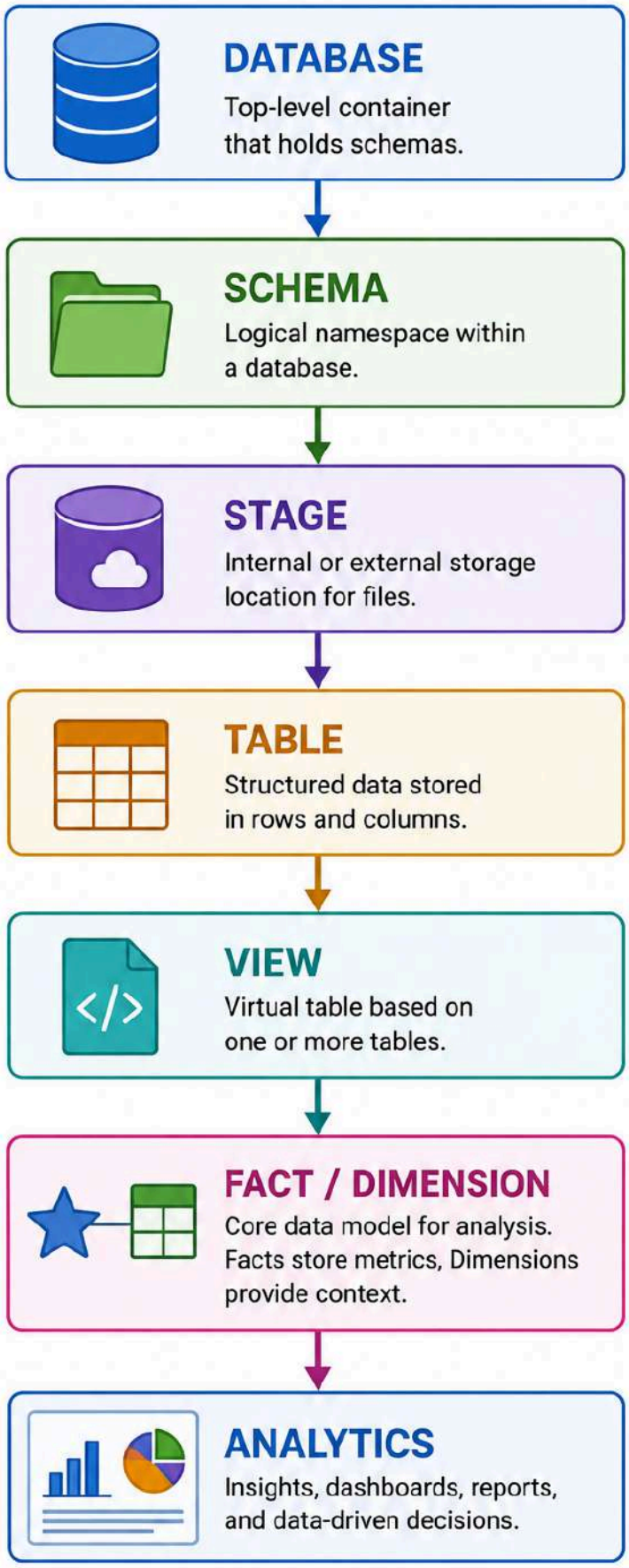

SOURCE SYSTEMS
ERP
CRM
API
APIs
DATA LAKE / RAW STORAGE
Store raw, unprocessed data
(Structured / Semi-structured / Unstructured)
STAGING
Clean, validate, and prepare data for processing
ENTERPRISE DATA WAREHOUSE
Integrated, cleansed, and historical data
SALES MART
Sales KPIs, Customer Insights, Performance Reports
FINANCE MART
Financial KPIs, Budgeting, Profitability Reports
HR MART
Employee KPIs, Workforce Analysis, HR Reports
BI / ANALYTICS
Dashboards • Reports • Data Visualization
Advanced Analytics • Machine Learning

---

50. Cloud Data Warehouse Comparison

Platform	Key Idea
Snowflake	Cloud-native data platform / warehouse
BigQuery	Serverless analytics warehouse
Redshift	AWS data warehouse
Databricks	Lakehouse/data + AI platform

For this course, **Snowflake** is particularly useful because it lets you practice:


DATABASE
Top-level container that holds schemas.
SCHEMA
Logical namespace within a database.
STAGE
Internal or external storage location for files.
TABLE
Structured data stored in rows and columns.
VIEW
Virtual table based on one or more tables.
FACT / DIMENSION
Core data model for analysis. Facts store metrics, Dimensions provide context.
ANALYTICS
Insights, dashboards, reports, and data-driven decisions.

51. Snowflake Lab — Recommended Learning Sequence

Use this sequence rather than jumping directly into complicated pipelines.

1. Snowflake UI	Access Snowflake web interface to manage resources and run queries.
2. Warehouse	Create and use a virtual warehouse to run queries and compute.
3. Database	Create a database to logically organize your data.
4. Schema	Create schemas inside a database to group database objects.
5. Table	Create tables to store structured data in rows and columns.
6. INSERT / SELECT	Insert data into tables and query data using SELECT.
7. Multiple databases	Use multiple databases to separate projects or domains.
8. Staging tables	Create staging tables to hold raw or intermediate data for processing.
9. File formats	Define file formats (CSV, JSON, Parquet, etc.) for loading data.
10. Stages	Create internal or external stages to store files in cloud storage.
11. COPY INTO	Load data from stages into tables using COPY INTO command.
12. Fact + Dimensions	Create fact and dimension tables for analytical modeling.
13. Star Schema	Design star schema with a central fact table and surrounding dimension tables.
14. SCD	Implement Slowly Changing Dimensions (Type 1, 2, 3) to track changes.
15. Data Mart	Build subject-oriented data marts from the enterprise data warehouse.
16. Analytical SQL	Write analytical SQL for reporting, dashboards, and insights.
17. Batch pipeline	Schedule and orchestrate batch jobs (ETL/ELT) to automate data workflows.

---

52. Batch Lab — End-to-End Exercise

Business Requirement

The company receives daily sales files.

sales_01.csv
sales_02.csv
sales_03.csv

Build:

CSV
↓
STAGE
↓
RAW TABLE
↓
DIMENSIONS
↓
FACT
↓
SALES MART
↓
REPORT

Student Tasks

Task 1

Create:

RAW_DB
DWH_DB
MART_DB

Task 2

Create:

RAW_DB.RAW
DWH_DB.STAGING
DWH_DB.DIMENSIONS
DWH_DB.FACTS
MART_DB.SALES

Task 3

Create:

CUSTOMERS
PRODUCTS
SALES

Task 4

Create:

DIM_CUSTOMER
DIM_PRODUCT
DIM_DATE
FACT_SALES

Task 5

Load the dimensions.

Task 6

Load the fact.

Task 7

Create:

SALES_SUMMARY

Task 8

Answer:

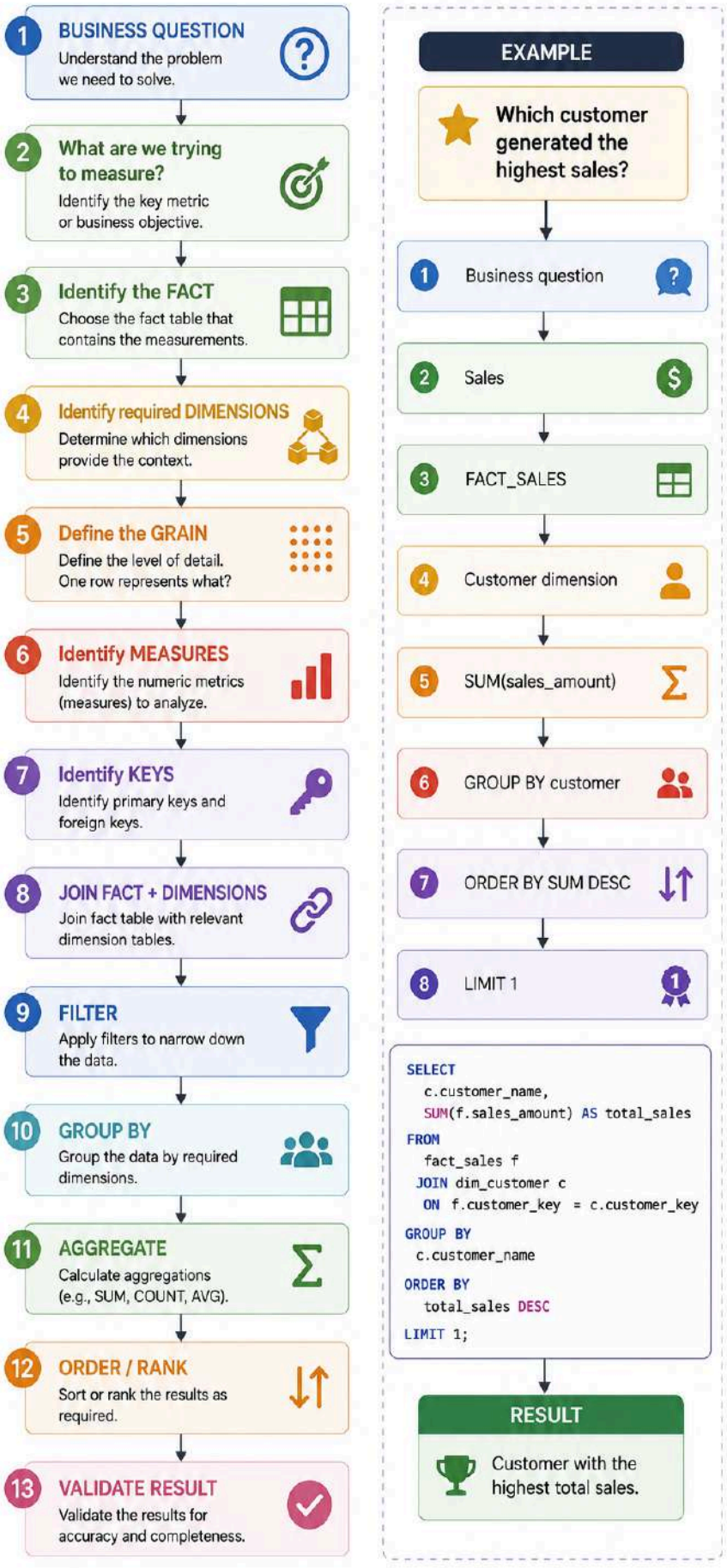
1. What is total sales?
2. Which customer generated the most sales?
3. Which product category generated the most revenue?
4. What was the daily sales trend?

5. Which customer bought the most quantity?
6. Which product generated the most revenue?

---

53. SQL Problem-Solving Framework for DWH

Use this framework whenever solving analytical SQL problems.


1 BUSINESS QUESTION
Understand the problem we need to solve.
2 What are we trying to measure?
Identify the key metric or business objective.
3 Identify the FACT
Choose the fact table that contains the measurements.
4 Identify required DIMENSIONS
Determine which dimensions provide the context.
5 Define the GRAIN
Define the level of detail. One row represents what?
6 Identify MEASURES
Identify the numeric metrics (measures) to analyze.
7 Identify KEYS
Identify primary keys and foreign keys.
8 JOIN FACT + DIMENSIONS
Join fact table with relevant dimension tables.
9 FILTER
Apply filters to narrow down the data.
10 GROUP BY
Group the data by required dimensions.
11 AGGREGATE
Calculate aggregations (e.g., SUM, COUNT, AVG).
12 ORDER / RANK
Sort or rank the results as required.
13 VALIDATE RESULT
Validate the results for accuracy and completeness.
EXAMPLE
Which customer generated the highest sales?
1 Business question
2 Sales
3 FACT_SALES
4 Customer dimension
5 SUM(sales_amount)
6 GROUP BY customer
7 ORDER BY SUM DESC
8 LIMIT 1
SELECT
c.customer_name,
SUM(f.sales_amount) AS total_sales
FROM
fact_sales f
JOIN dim_customer c
ON f.customer_key = c.customer_key
GROUP BY
c.customer_name
ORDER BY
total_sales DESC
LIMIT 1;
RESULT
Customer with the highest total sales.

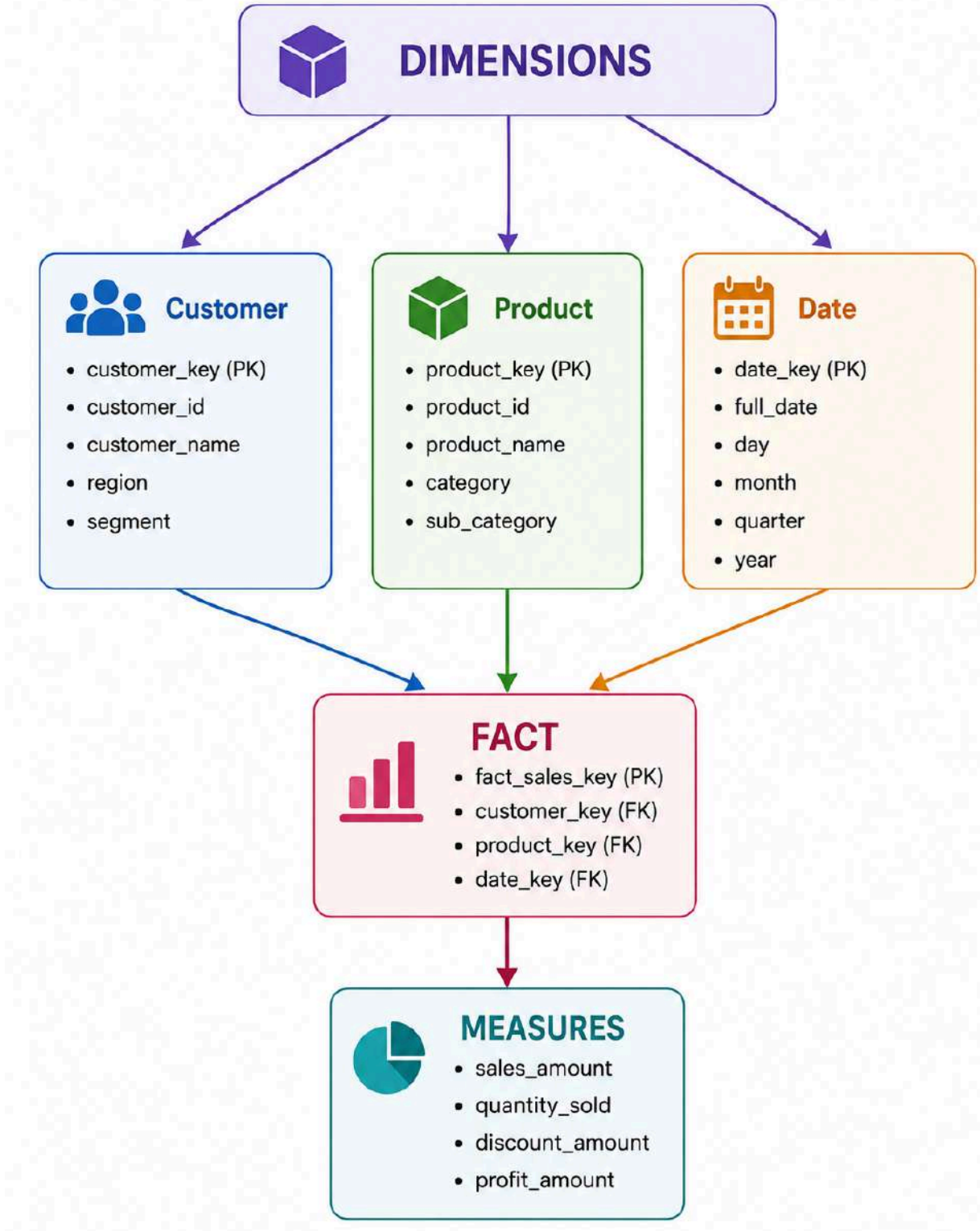
SQL:

```
SELECT
   c.CUSTOMER_NAME,
   SUM(f.SALES_AMOUNT) AS TOTAL_SALES
FROM DWH_DB.FACTS.FACT_SALES f
JOIN DWH_DB.DIMENSIONS.DIM_CUSTOMER c
   ON f.CUSTOMER_KEY = c.CUSTOMER_KEY
GROUP BY c.CUSTOMER_NAME
ORDER BY TOTAL_SALES DESC
LIMIT 1;
```

---

54. Most Important DWH Mental Model

Remember:



Dimensions answer:

WHO?
WHAT?
WHEN?
WHERE?

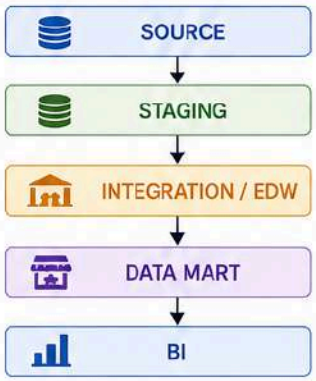
Facts answer:

HOW MANY?
HOW MUCH?

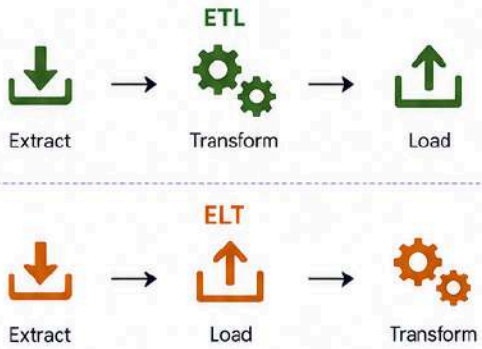
---

55. DWH Cheat Sheet

ARCHITECTURE



ETL vs ELT



FACT

FACT = EVENTS + MEASURES + FOREIGN KEYS

Example: FACT_SALES

EVENTS	MEASURES	FOREIGN KEYS
• sale_date	• quantity_sold	• customer_key
• order_id	• sales_amount	• product_key
• line_number	• discount_amount	• date_key
	• profit_amount	

DIMENSION

DIMENSION = DESCRIPTIVE CONTEXT

Example: DIM_CUSTOMER

- customer_key (PK)
- customer_name
- region
- segment
- city
- country
- ...

GRAIN

GRAIN = WHAT ONE ROW REPRESENTS

Example

One row in FACT_SALES represents one **product** sold in one **order** on a specific **date** by one **customer**.

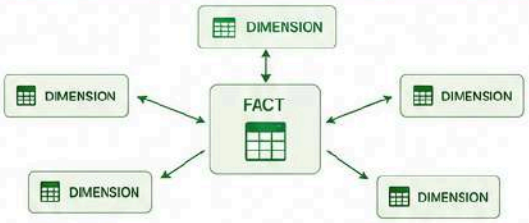
SURROGATE KEY

SURROGATE KEY = WAREHOUSE-GENERATED IDENTIFIER

Example

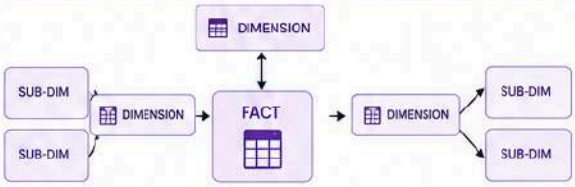
- Does not carry business meaning
- Integer / sequence
- Stable even if source key changes
- Used as Primary Key in DW

STAR SCHEMA



FACT in the center connected to denormalized dimensions.

SNOWFLAKE SCHEMA



FACT connected to normalized dimensions which are further broken into sub-dimensions.

SCD (SLOWLY CHANGING DIMENSIONS)

TYPE 1 – OVERWRITE

Before Update				After Update		
Customer Key	Name	City	→			
1	ABC Corp	Delhi		1	ABC Corp	Mumbai

Existing row is updated.
History is lost.

TYPE 2 – NEW ROW

Before Update			After Update		
Customer Key	Name	City	Start Date	End Date	Current
1	ABC Corp	Delhi	2022-01-01	9999-12-31	Y
↓					
1	ABC Corp	Delhi	City	2022-01-01	2024-04-30	N
↓					
1	ABC Corp	Mumbai	Delhi	2024-05-01	9999-12-31	Y

New row is inserted for changes.
Full history is preserved.

TYPE 3 – NEW COLUMN

Before Update			After Update			
Customer Key	Name	City	Customer Key	Name	City	City Previous
1	ABC Corp	Delhi	1	ABC Corp	Mumbai	Delhi

New column added to store previous value.
Limited history (only last change).

56. Snowflake SQL Cheat Sheet

Database

```
CREATE DATABASE DB_NAME;
```

Schema

```
CREATE SCHEMA DB_NAME.SCHEMA_NAME;
```

Select database

```
USE DATABASE DB_NAME;
```

Select schema

```
USE SCHEMA SCHEMA_NAME;
```

Fully qualified table

```
DATABASE.SCHEMA.TABLE
```

Create table

```
CREATE TABLE DB.SCHEMA.TABLE_NAME (
   ID INT,
   NAME VARCHAR(100)
);
```

Insert

```
INSERT INTO DB.SCHEMA.TABLE_NAME
VALUES (1, 'Alice');
```

Select

```
SELECT *
FROM DB.SCHEMA.TABLE_NAME;
```

Stage

```
CREATE STAGE STAGE_NAME;
```

File format

```
CREATE FILE FORMAT CSV_FORMAT
```

```
TYPE = CSV
SKIP_HEADER = 1;
```

Load

```
COPY INTO TABLE_NAME
FROM @STAGE_NAME
FILE_FORMAT = (FORMAT_NAME = CSV_FORMAT);
```

Snowflake's documented loading workflow uses stages/file formats together with `COPY INTO`; local files are typically staged first, while cloud storage can also be used through external stages.

---

57. One-Page Exam Revision

DATA WAREHOUSE

Centralized, integrated, subject-oriented, time-variant, non-volatile repository for analytical reporting and decision making.

Architecture

- Staging – Temporary area for raw data
- Integration – Clean, transform and integrate data
- Access – Data marts / reporting layer for business users

EDW
(Enterprise Data Warehouse)

- Enterprise-wide data repository
- Integrates data from across the organization
- Single version of truth

DATA MART
(Business-area focused)

- Department / business unit specific (Sales, Finance, HR, etc.)
- Subset of EDW data
- Faster and simpler for analytics

ETL
(Extract – Transform – Load)

- Transform data **before** loading into the warehouse
- Used in traditional on-premise data warehousing

ELT
(Extract – Load – Transform)

- Load data **before** transformation
- Transform inside the data warehouse
- Popular with cloud data warehouses (Snowflake, BigQuery, Redshift)

BATCH
(Periodic processing)

- Data processed at scheduled intervals
- Daily / hourly / weekly loads
- Cost effective, high throughput

REAL-TIME
(Continuous / low-latency)

- Continuous or near real-time data processing
- Low latency, up-to-date insights
- Used for operational analytics / live dashboards

DATA MODELING
(Structure data for analytical queries)

FACT

- Stores numeric measures / metrics
- Example: Sales Amount, Quantity

DIMENSION

- Stores descriptive attributes
- Example: Customer, Product, Date

GRAIN

- Defines the level of detail
- One row represents... (e.g., one sale per product per day)

SURROGATE KEY

- Warehouse-generated artificial key
- No business meaning, ensures uniqueness

STAR

- Denormalized dimensions
- Fewer joins, simpler and faster queries

SNOWFLAKE

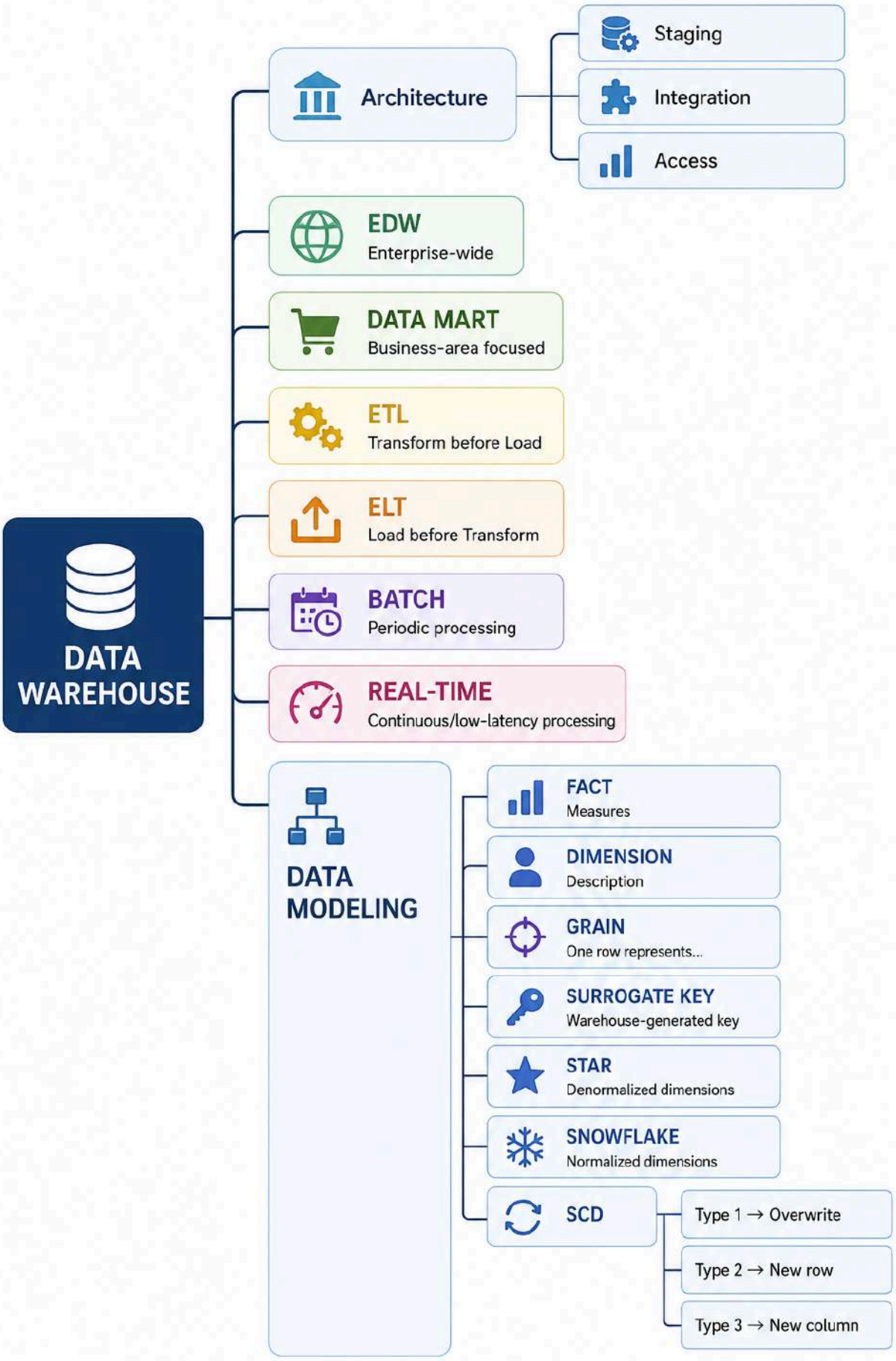
- Normalized dimensions
- More joins, less redundancy

SCD
(Slowly Changing Dimensions)

- **Type 1** → Overwrite existing value (no history)
- **Type 2** → New row for changed value (full history)
- **Type 3** → New column for old and new value

Key Benefits

- ✓ Single source of truth
- ✓ Better decision making
- ✓ Historical analysis
- ✓ Consistent and integrated data
- ✓ Scalable for future growth


DATA WAREHOUSE
Architecture
Staging
Integration
Access
EDW
Enterprise-wide
DATA MART
Business-area focused
ETL
Transform before Load
ELT
Load before Transform
BATCH
Periodic processing
REAL-TIME
Continuous/low-latency processing
DATA MODELING
FACT
Measures
DIMENSION
Description
GRAIN
One row represents...
SURROGATE KEY
Warehouse-generated key
STAR
Denormalized dimensions
SNOWFLAKE
Normalized dimensions
SCD
Type 1 → Overwrite
Type 2 → New row
Type 3 → New column

---

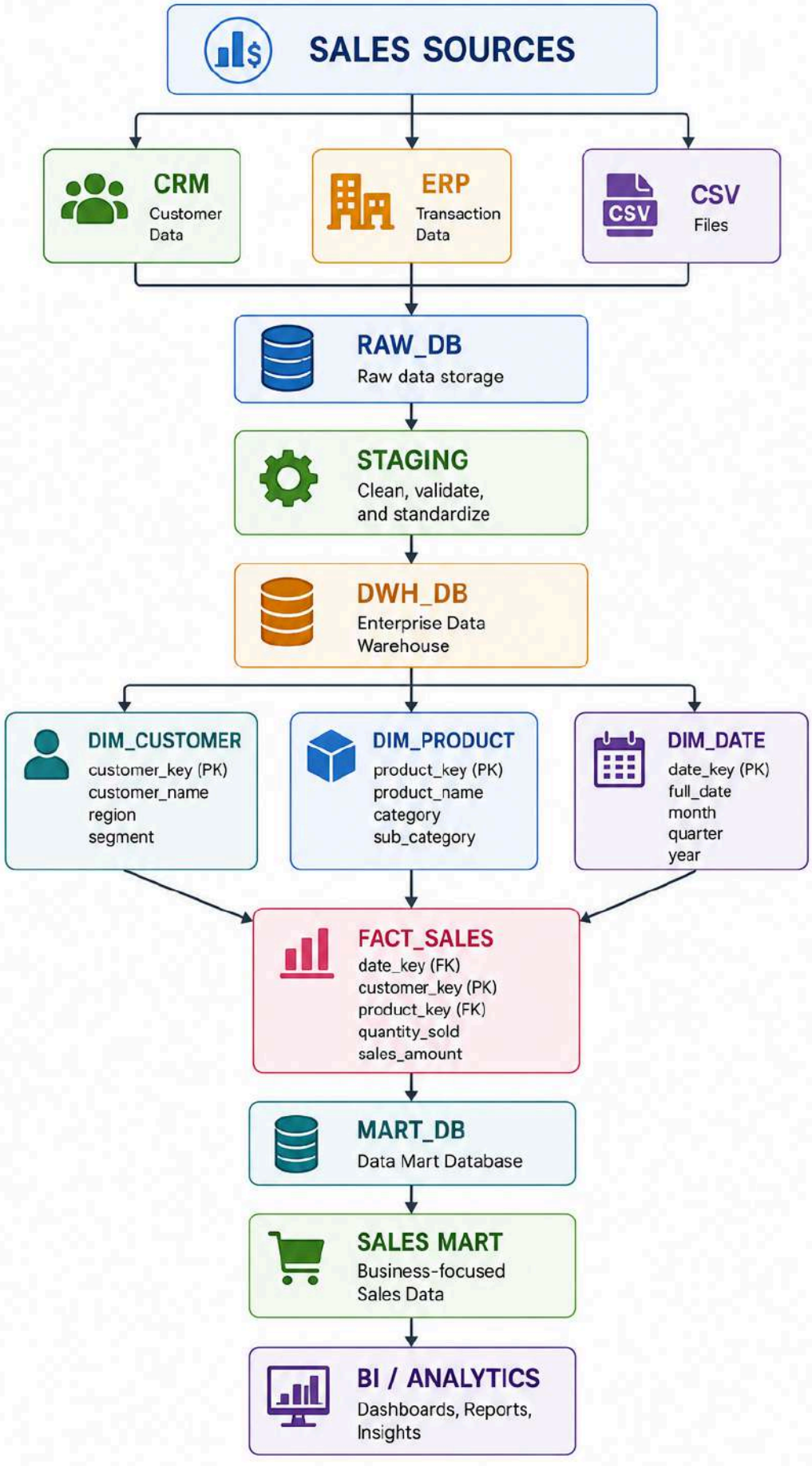
58. Suggested Classroom Lab Structure

Lab	Topic	Hands-on
Lab 1	DWH Architecture	Draw architecture
Lab 2	Batch Processing	CSV → Stage → Table
Lab 3	Snowflake Basics	Database + Schema + Table
Lab 4	Multiple Databases	RAW → DWH → MART
Lab 5	Dimensions	Customer/Product/Date
Lab 6	Fact Table	Fact Sales
Lab 7	Star Schema	Complete ERD
Lab 8	Snowflake Schema	Normalize dimensions
Lab 9	SCD Type 1	UPDATE
Lab 10	SCD Type 2	Historical rows
Lab 11	SCD Type 3	Previous-value column
Lab 12	Analytics	GROUP BY + JOIN
Lab 13	Data Mart	Sales summary
Lab 14	End-to-End	Source → BI

---

59. Final Capstone

The best final exercise.


SALES SOURCES
CRM
Customer Data
ERP
Transaction Data
CSV
Files
CSV
RAW_DB
Raw data storage
STAGING
Clean, validate, and standardize
DWH_DB
Enterprise Data Warehouse
DIM_CUSTOMER
customer_key (PK)
customer_name
region
segment
DIM_PRODUCT
product_key (PK)
product_name
category
sub_category
DIM_DATE
date_key (PK)
full_date
month
quarter
year
FACT_SALES
date_key (FK)
customer_key (PK)
product_key (FK)
quantity_sold
sales_amount
MART_DB
Data Mart Database
SALES MART
Business-focused Sales Data
BI / ANALYTICS
Dashboards, Reports, Insights

The final student should be able to explain the entire pipeline as:

> **Data comes from source systems, lands in staging, gets integrated into the warehouse, is modeled using facts and dimensions, and is exposed through data marts for analytics.**

For Snowflake specifically, the core object hierarchy to remember is **database** → **schema** → **table**, and the three-part name `database.schema.table` lets you explicitly reference objects across databases.

Links

Database, schema, & share DDL | Snowflake Documentation
COPY INTO *<table>* | Snowflake Documentation
Copy data from an internal stage | Snowflake Documentation
https://docs.snowflake.com/en/user-guide/data-load-local-file-system-stage